

Projeto e implementação de compilador - parte 1

Projete e implemente um analisador léxico para a linguagem C-- definida pela gramática livre de contexto presente no fim do documento

1. Liste o conjunto de tokens da linguagem. Eles serão reconhecidos e retornados pelo seu analisador léxico
2. Derive ou desenhe um único AFD que reconheça todos os tokens
3. Implemente o AFD em linguagem C++ ou Python. O código deve ser claro
4. Implemente uma tabela de símbolos para gerenciar e diferenciar palavras reservadas, identificadores e literais

Documentação:

A documentação a ser entregue deve ser objetiva e completa, explicando os detalhes de como o programa funciona e as conclusões obtidas pelo trabalho.

Além disso, tudo que for feito e apresentado na documentação deve ter referência de artigos ou livros. No caso de livros seja específico e indique a seção. Este documento deve conter:

1. Título
2. Visão geral (Introdução)
 - a. Funcionamento geral do programa
 - b. Decisões de implementação tomadas
 - c. Breve descrição de cada módulo do programa
3. Tokens e expressões regulares (as expressões regulares são opcionais)
 - a. Apresente uma tabela de tokens como exemplo, indicando seus respectivos lexemas e atributos. Mostre as palavras reservadas da linguagem
 - b. Tokens são representados como valores inteiros internamente no programa. Liste os tokens com os valores que escolheu
 - c. Exiba o AFD que reconhece os tokens definidos
4. Analisador léxico
 - a. Descreva a forma como implementaram o analisador léxico
 - b. Apresente as estruturas de dados, os métodos e os algoritmos utilizados
 - c. Exiba o AFD desenhado
5. Tabela de símbolos

- a. Descreva a forma como implementaram a classe de tabela de símbolos
 - b. Apresente as estruturas de dados, os métodos e os algoritmos utilizados
6. Gerenciador de erros léxicos
 - a. Apresente claramente um método para identificar e apresentar onde foram encontrados erros
7. Resultados experimentais
 - a. Apresente testes feitos e resultados encontrados pelo grupo
 - b. Indique testes que deram certo e testes que contém erros
8. Conclusão
9. **Referências Bibliográficas**
 - a. Mostre todas as fontes que vocês usaram para fazer o trabalho. **Artigos, livros, links** e outros materiais usados por vocês devem ser referenciados mas não precisam seguir nenhuma norma específica para citação desde que seja clara a origem do documento. Para **capítulos de livros indique quais seções específicas foram usadas em cada etapa do trabalho**. Todas as seções do trabalho devem ser acompanhadas de ao menos três referências.

Especificações para o analisador léxico

- Utilize a bibliografia disponibilizada para desenvolver o trabalho
- Permita, no seu programa, que seja passado como parâmetro por linha de comando o arquivo de entrada para fazer os testes
- As strings numéricas e alfanuméricas não podem ter restrição de comprimento. Para isso, utilize um arranjo dinâmico de caracteres, ou bibliotecas prontas com essa funcionalidade, como vector
- O analisador deve desconsiderar comentários escritos no programa fonte. A única notação para comentários na linguagem c– é:
/* comentário aqui */
- O gerenciador de erros não deve interromper a execução do analisador léxico. Ao contrário, ao encontrar um erro, indique a linha e coluna em que aconteceu o erro, apresente uma mensagem indicando qual foi o erro de forma clara para o usuário e então prossiga buscando o próximo token

- Ao chegar no final do arquivo o Analisador léxico deve retornar o caractere especial **EOF**
- A saída do analisador léxico deve ser a sequência de tokens encontrados. Para facilitar a correção, não imprima a versão numérica de cada token. Ao invés disso, imprima o nome de cada token. Por exemplo

A entrada: `if(x1 <= 32) b = 10;`

Deve ser convertida na saída:

IF

LPARENT

ID.x1

LEQ

NUM.32

RPARENT

ID.b

ASSING

NUMINT.10

SEMICOLON

EOF

- A tabela de símbolos pode ser feita como um vetor de busca de ordem linear, um vetor de busca binária, um hash ou uma árvore TRIE ou PATRICIA
- Todas as strings reconhecidas pelo autômato devem ser pesquisadas na tabela de símbolos para descobrir seu token

Desenvolvendo o trabalho

- Encontre os tokens da linguagem a partir da gramática no final do documento
- Espaços e comentários não geram tokens. Eles devem ser descartados a menos que componham uma string
- Procure determinar tokens individuais para cada símbolo de pontuação e para operadores (COMMA para ‘,’ , SEMICOLON para ‘;’, GEQ para “>=”). A outra opção, não recomendada, é definir um token para todos os símbolos e diferenciá-los através de um atributo

Gramática C-

Program -> FunctionDecl Program |
 TypeDecl Program |
 VarDecl Program |
 FunctionDecl

TypeDecl --> **typedef struct** {Type IdList ; VarDecl} ;
TypeDecl | epsilon

VarDecl -> Type idList ; VarDecl | epsilon

IdList -> **ID** Array |
 IdList, **ID** Array

Array -> [**NUM**] | epsilon

FunctionDecl -> Type **ID** (FormalList) { VarDecl
StmList }

FormalList -> Type **ID** Array FormalRest | epsilon

FormalRest -> , Type **ID** Array FormalRest | epsilon

Type -> **int** | **float** | **bool** | **ID** | **char**

StmList -> Stmt | Stmt StmList

Stmt -> **if** (Expr) Stmt **else** Stmt |
 while (Expr) Stmt |
 break ; |

```

print ( ExprList ) ; |
readln ( Expr ) ; |
return Expr ;|
{ StmtList } |
ID ( ExprList ) ; |
Expr ;

```

ExprList -> epsilon | ExprListTail

ExprListTail -> Expr | Expr , ExprListTail

```

Expr -> Primary |
      UnaryOp Expr |
      Expr BinOp Expr |
      Expr = Expr

```

```

Primary -> ID | NUM | LITERAL |
          ' ASCII ' |
          ( Expr ) |
          ID ( ExprList ) |
          Expr [ Expr ] |
          true | false

```

UnaryOp -> -- | ++ | !

```

BinOp ->  == | < | <= |
          >= | > | != | + | - |
          || | * | / | % | &&

```

